

Jovica Filipović 19455
Vukašin Stamenković 19364

Internet stvari i servisa

IoT Mikroservisi zasnovani na događajima: MQTT vs Kafka

Jun, 2026

1. Uvod

Cilj projekta je istraživanje performansi, skalabilnosti i ograničenja message broker sistema zasnovanih na publish-subscribe modelu u IoT mikroservisnim arhitekturama. Sistem je implementiran kao asinhroni, event-driven mikroservisni sistem koji koristi dva brokera – MQTT (Mosquitto) i Apache Kafka – nad istim IoT datasetom senzorskih podataka.

2. Tehnologije

Data Ingestion Service – Node.js / NestJS

Analytics Service – Node.js / NestJS

Data Storage Service – ASP.NET Core (.NET 8)

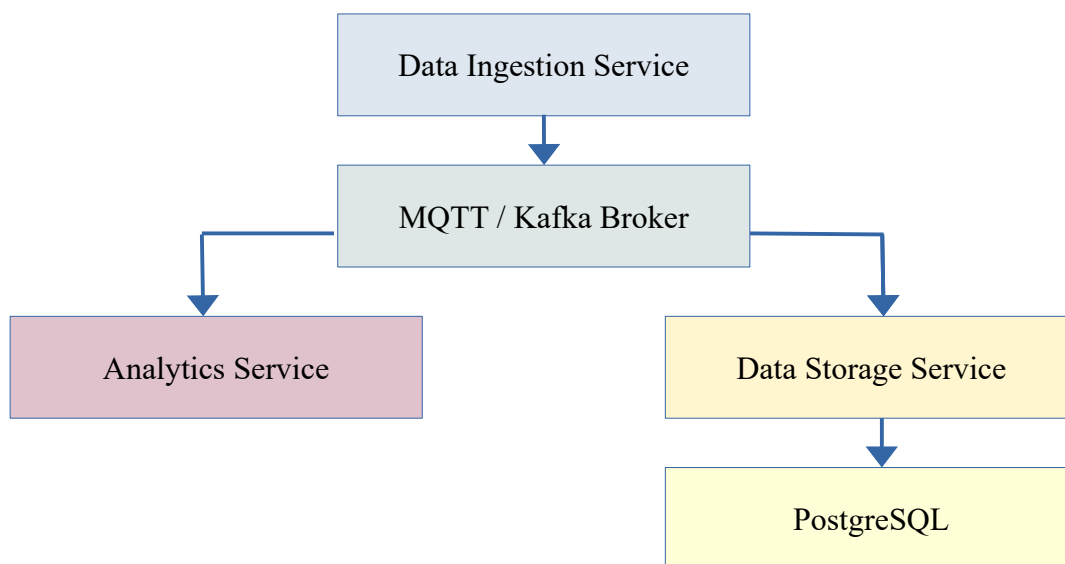
Message Broker 1 - Eclipse Mosquitto (MQTT)

Message Broker 2 - Apache Kafka

Baza podataka – PostgreSQL

3. Arhitektura sistema

Sistem se sastoji od tri mikroservisa koji komuniciraju isključivo kroz message broker – bez direktne međusobne komunikacije:



Data Ingestion Service simulira N IoT uređaja koji paralelno generišu senzorske podatke (temperatura, vlažnost, CO, dim, svetlost) u nasumičnim vrednostima unutar opsega realnog dataseta i šalju ih na broker.

Analytics Service implementira Tumbling Window od 10 sekundi – svakih 10 sekundi izračunava prosečnu temperaturu primljenih poruka i ispisuje kritičan alarm ako prosek prelazi 50°C.

Data Storage Service prima poruke sa brokera i upisuje ih u PostgreSQL bazu podataka kroz batch mehanizam – grupni upis na svakih 500 poruka ili svakih 5 sekundi, što pre nastupe.

4. Konfiguracija

- MQTT konfiguracija

Mosquitto broker je konfigurisan kroz **config/mosquitto.conf**:

- Port: 1883
- Anonimni pristup dozvoljen
- Persistencija poruka omogućena
- CleanSession=false na Storage servisu (čuva poruke za offline klijente)

Nivo kvaliteta usluge (QoS) se konfigurišu kroz environment varijable:

- MQTT_QOS – na Ingestion servisu (publish strana)
- Broker__Mqtt__QoS – na Storage servisu (subscribe strana)

- Kafka konfiguracija

Kafka radi u KRaft režimu (bez Zookeeper-a) što značajno smanjuje memorijski footprint. Konfiguracija:

- Port: 9092
- 3 particije po topiku
- Auto-kreiranje topica omogućeno
- Replication factor: 1 (single broker setup)

Parametri potvrde prijema konfigurišu se kroz KAFKA_ACKS environment varijablu na Ingestion servisu (0, 1, ili -1 za all).

- Prebacivanje izmedju brokera

Svi servisi podržavaju oba brokera. Prebacivanje se vrši isključivo kroz environment varijable u **docker-compose.yml** bez izmene koda:

BROKER_TYPE: mqtt ili kafka

- Pokretanje sistema

Kompletan sistem se pokrece preko dockera: `docker compose up -d`

5. Eksperimentalni scenariji i rezultati merenja

1. QoS testiranje (MQTT) — kombinovano sa Scenario B (network disconnect)

Pošto u normalnim uslovima (bez gubitka paketa) QoS 0/1/2 daju identičan rezultat (sve poruke stižu), test je osmišljen da izazove gubitak diskonektovanjem iot-storage od mreže na 30s (Scenario B).

Setup:

BROKER_TYPE=mqtt

MQTT_QOS=0 / 1 / 2

Broker__Mode=mqtt

Broker__Mqtt__QoS=1

DEVICE_COUNT=3

INTERVAL_MS=2000

QoS	Poruka stiglo	Catch-up skok nakon reconnect-a	Zakljucak
0	146	Ne	Poruke tokom 30s prekida trajno izgubljene – broker ih nije čuvao
1	73	Da (jedan batch od 87 poruka)	Broker je zadržao poruke i isporučio ih odjednom.
2	89	Da (jedan batch od 90 poruka)	Broker je zadržao poruke i isporučio ih odjednom.

2. Acks testiranje (Kafka) — kombinovano sa Scenario B (network disconnect) i Consumer Lag

Testiranje je sprovedeno na isti način kao i MQTT QoS testiranje – za svaki acks nivo simuliran je mrežni prekid od 30 sekundi isključivanjem Data Storage servisa iz mreže, nakon čega je praćen oporavak sistema. Za razliku od MQTT testa gde je fokus bio na gubitku poruka, ovde je dodatno praćen *Consumer Lag* - metrika koja pokazuje koliko poruka čeka u Kafka topicu a nije još pročitano od strane Storage servisa.

Setup:

BROKER_TYPE=kafka

KAFKA_ACKS=0 / 1 / -1

Broker__Mode=kafka

acks	Poruka stiglo	Lag PRE	Lag Tokom	Lag Posle	Izgubljene poruke
0	420	6	24	3	-
1	122	6	27	3	-
-1 (all)	17	6	21	3	-

3. Scenario A – Massive Sensor Ingestion (DEVICE_COUNT)

Simulirati paralelni rad 100, 1000 i 10000 uređaja. Pratiti maksimalni throughput (broj poruka u sekundi) i procenat izgubljenih poruka.

Setup:

BROKER_TYPE=kafka

KAFKA_ACKS=1

DEVICE_COUNT=100 / 1000 / 10000

INTERVAL_MS=2000

Br. uređj.	Poruka stiglo	Ocekivno poruka za 60s	Throughput (msg/s)	CPU %	RAM (MiB)
100	3333	3000	54	3,03 - Ingestion 2,36 - Storage 4,07 - Kafka	58,7- Ingestion 45,1 - Storage 520,9 - Kafka
1000	34000	30000	512	10,51 - Ingestion 14,04 - Storage 58,24 - Kafka	67,4- Ingestion 53,1 - Storage 934 - Kafka
10000	/	300000	/	/	/

Test za 10000 uređaja nije uspešno izvršen zbog hardverskih limita računara.

4. Scenario B – (Edge Connectivity Failures)

Pomoću komande `docker network disconnect` simulirati mrežni prekid na simulatoru uređaja u trajanju od 30 sekundi. Pratiti recovery mehanizme oba brokera nakon ponovnog povezivanja (trajne pretplate kod MQTT-a vs. pomeranje `offset-a` kod Kafke).

Tabele Testova 1. i 2.

5. Scenario C – (Burst Event Load)

Generisati nagli skok sa 50 na 5000 poruka/s u trajanju od nekoliko sekundi. Pratiti formiranje reda čekanja (backlog), backpressure ponašanje i vreme potrebno da se sistem vrati u normalu (recovery time).

Setup:

BROKER_TYPE=kafka

KAFKA_ACKS=1

INTERVAL_MS=2000

Fazs	Broj uređaja	Poruka stiglo	Throughput (msg/s)	Consumer Lag	Napomena
Baseline	100	3920	134	173	
Burst	2000	39780	1011	2498	Backlog formation potvrđen, lag naglo raste kad producer nadmaši consumer throughput
Recovery	100	-	-	100	

6. Scenario D – (Real-Time Alerting)

Izmeriti end-to-end latenciju – vreme od trenutka kada simulator generiše kritičnu vrednost do trenutka kada Analytics Service ispiše alarm.

Setup:

BROKER_TYPE=kafka

DEVICE_COUNT=100

INTERVAL_MS=2000

WINDOW_MS=10000

TEMP_THRESHOLD=50°C

Pokusaj	Vreme (window log)	Avg Temp	Alert Ispisan	Vreme (alert log)	Latencija
1	8:41:14 PM	50.81	Da	8:41:14 PM	0s
2	8:41:36 PM	51.13	Da	8:41:36 PM	0s
3	8:41:51 PM	50.68	Da	8:41:51 PM	0s
4	8:42:03 PM	51.61	Da	8:42:03 PM	0s

7. Benchmark skripte MQTT

Za MQTT testiranje korišćen je emqtt-bench alat koji meri propusnost (poruka/sekundi) i broj izgubljenih poruka.

Skripte se nalaze u folderu **benchmarks/mqtt**.

QoS	CPU %	RAM MiB	NET I/O MB
0	33.65 – MQTT 16.42 – Ingestion 11.49 – Analytics 3.08 - Storage	4.758 – MQTT 60.76 – Ingestion 62.48 – Analytics 59.46 - Storage	652 / 101– MQTT 10.9 / 28.1– Ingestion 36.6 / 26.2 – Analytics 38.8 / 24.9 - Storage
1	58.88 – MQTT 16.78 – Ingestion 23.32 – Analytics 7.00 - Storage	6.08 – MQTT 63.09 – Ingestion 64.83 – Analytics 64.83 - Storage	1.56GB / 274 – MQTT 13.7 / 35– Ingestion 46.5 / 33.5 – Analytics 48.9 / 31.3 - Storage
2	72.23 – MQTT 6.14 – Ingestion 7.70 – Analytics 4.46 - Storage	5.80 – MQTT 63.36 – Ingestion 64.56 – Analytics 63.93 - Storage	1.99GB / 390– MQTT 15.2 / 39– Ingestion 51.3 / 36.9 – Analytics 54 / 34.5 - Storage

Emqtt-bench alat ne podržava merenje latencije.

8. Benchmark skripte Kafka

Za Kafka testiranje korišćen je nativan kafka-producer-perf-test.sh

Skripte se nalaze u folderu **benchmarks/kafka**.

acks	CPU %	RAM MiB	NET I/O MB	p95 ms
0	145.6 – Kafka 7.62 – Ingestion 73.52 – Analytics 71.58 - Storage	1.09Gib – Kafka 64.18 – Ingestion 83.6 – Analytics 90.96 - Storage	158 / 285 – Kafka 19.9 / 51.8 – Ingestion 130 / 50.2 – Analytics 135 / 46 - Storage	968.28
1	159.2 – Kafka 11.2 – Ingestion 44.58 – Analytics 73.37 - Storage	1.12GiB – Kafka 64.49 – Ingestion 69.46 – Analytics 80.33 - Storage	141 / 332 – Kafka 20 / 54 – Ingestion 154 / 52.5 – Analytics 157 / 48.1 - Storage	2283.7
all	161.5 – Kafka 17.8 – Ingestion 58.6 – Analytics 81.13 - Storage	1.178Gib – Kafka 64.42 – Ingestion 69.12 – Analytics 78.18 - Storage	148 / 422 – Kafka 21 / 56.4 – Ingestion 198 / 54.8 – Analytics 202 / 50.5 - Storage	1982

6. Analiza pouzdanosti i garancija isporuke

6.1. Zašto je MQTT idealan za edge uređaje, a neadekvatan za istorijsku analitiku?

MQTT je protokol dizajniran specifično za okruženja sa ograničenim resursima i nestabilnim mrežama. Nekoliko karakteristika ga čine idealnim za edge: Mosquitto broker može raditi na uređaju sa svega 256MB RAM-a, a sam protokol ima header od samo 2 bajta što ga čini baterijsko-efikasnim. Takođe, QoS mehanizmi (0, 1, 2) omogućavaju prilagođavanje garancija isporuke karakteristikama mreže – na nestabilnim vezama QoS 1 ili 2 garantuje da poruka stigne do brokera čak i uz privremene prekidne. Treće, CleanSession=false mehanizam omogućava brokeru da čuva poruke za offline klijente i isporuči ih pri ponovnom povezivanju.

Međutim MQTT postaje neadekvatan za istorijsku analitiku velikih podataka iz fundamentalnih arhitekturnih razloga. MQTT broker je posrednik koji prosleđuje poruke – ne čuva ih trajno. Kada poruka bude isporučena svim subscriberima, ona nestaje. Ne postoji mogućnost "replay-a" istorijskih poruka, ne postoji particionisanje koje bi omogućilo paralelno čitanje, a sam broker postaje usko grlo pri milionima poruka jer ne može horizontalno da se skalira. Ovo je potvrđeno i u našim testovima, kod QoS 0 tokom 30-sekundnog mrežnog prekida sve poruke su trajno izgubljene jer broker nije imao mehanizam za njihovo čuvanje.

6.2. Zašto Kafka dominira u data-intensive cloud sistemima i kolika je cena?

Zbog toga što Kafka je arhitekturno fundamentalno drugačija od MQTT-a – nije broker koji prosleđuje poruke, već distribuirani persistent log. Svaka poruka se upisuje na disk i čuva konfigurisani period (retention), što omogućava bilo kom consumeru da "pročita istoriju" od proizvoljne tačke. Particionisanje omogućava paralelno čitanje od strane više consumer instanci, consumer grupe omogućavaju da više servisa nezavisno konzumira isti topic, a throughput može dostići milione poruka u sekundi horizontalnim skaliranjem.

Ovo je potvrđeno u Scenariju B – za razliku od MQTT QoS 0, Kafka nije izgubila nijednu poruku tokom 30-sekundnog prekida bez obzira na acks nivo, jer topic čuva poruke nezavisno od consumer-a, a offset-based model garantuje nastavak od tačno mesta gde je consumer stao. Kafka zahteva više konfiguracije, ima veću operativnu kompleksnost i veću latenciju od MQTT-a zbog persistencije na disk.